Router par la preuve et s'abstenir à temps : allocation adaptative du calcul et de la recherche documentaire pour des LLM locaux

Une évaluation pré-enregistrée de la qualité, de la latence et de l'énergie sur un ordinateur personnel

Alae Lahbichi

Étudiant en 5e année, Intelligence artificielle et science des données

a.lahbichi2004@gmail.com

Code et données : github.com/AlaeLahbichi/Adaptive-Compute-and-RAG-for-Efficient-Reliable-Local-LLMs

Octobre 2026

Résumé

Faire tourner un modèle de langage sur sa propre machine, c'est garder la maîtrise de ses données, mais c'est aussi payer chaque réponse en secondes d'attente et en joules. Toutes les questions ne méritent pourtant pas le même effort : certaines se règlent sans aucun document, d'autres demandent un modèle plus puissant et plusieurs passages, d'autres encore n'ont pas de réponse dans le corpus, et la seule réponse honnête est alors de s'abstenir. Cet article étudie comment prendre ces trois décisions ensemble, question par question, à partir de signaux qui ne coûtent presque rien. Nous proposons une politique qui s'appuie sur la suffisance des preuves, mesurée par le score BM25 du meilleur passage, et sur la confiance du modèle, lue dans les probabilités des tokens qu'il produit. Nous l'évaluons sur un Apple M3 Pro avec deux modèles Qwen quantifiés en 4 bits, sur un benchmark de 630 questions construit à partir de SQuAD 2.0 et HotpotQA, en fixant les critères de succès avant toute expérience et en n'exécutant le jeu de test qu'une seule fois. Sur 450 questions de test, la politique égale la qualité de la configuration de référence (score de 0,550 contre 0,537, non-infériorité démontrée), commet moins d'erreurs et devance deux routeurs inspirés de la littérature. Elle ne réduit cependant la latence que de 10 % et l'énergie de 9 %, quand nous visions 20 % : le critère principal n'est pas atteint. Nous montrons que cet écart tient au réglage d'un seuil absolu sur un échantillon trop petit, et nous en tirons un programme d'améliorations précis.

Mots-clés : modèles de langage locaux, routage de modèles, RAG adaptatif, abstention, sobriété énergétique, évaluation pré-enregistrée.

1 Introduction

1.1 Contexte

Il y a encore peu, interroger un grand modèle de langage supposait d'envoyer sa question à un serveur distant. Ce n'est plus le cas. Des modèles de quelques milliards de paramètres, une fois quantifiés, tiennent dans la mémoire d'un ordinateur portable et y répondent en moins d'une seconde. Cette évolution change la nature du problème. Une entreprise qui veut interroger ses archives sans les confier à un tiers, un chercheur qui travaille sur des données sensibles, un étudiant qui n'a pas de budget pour une API : tous peuvent désormais disposer d'un assistant local. En contrepartie, le coût de chaque réponse cesse d'être une ligne abstraite sur une facture. Il devient du temps d'attente, de la mémoire occupée, de la batterie consommée.

Deux leviers permettent d'améliorer les réponses d'un tel système, et tous deux ont un prix. Le premier est la taille du modèle : un modèle plus grand raisonne mieux, mais il est plus lent.

Le second est la recherche documentaire, ou génération augmentée par récupération (RAG) : en joignant à la question des passages extraits d'un corpus, on ancre la réponse dans des sources et l'on réduit les inventions du modèle (Gao et al., 2023). Mais chaque passage ajouté allonge le texte que le modèle doit lire, et donc le temps de calcul. Il existe enfin une troisième issue, souvent oubliée : ne pas répondre. Lorsque les documents ne contiennent pas l'information demandée, une abstention franche vaut mieux qu'une réponse plausible et fausse (Wen et al., 2024).

1.2 Problématique

La pratique courante consiste à fixer une configuration une fois pour toutes, par exemple « le plus grand modèle disponible avec trois passages », et à l'appliquer à toutes les questions. Ce choix est confortable, mais il est doublement inefficace. Sur une question simple, dont la réponse figure en toutes lettres dans un passage, il mobilise un modèle surdimensionné. Sur une question à laquelle le corpus ne répond pas, il dépense une génération complète pour produire, au mieux, un aveu d'ignorance et, au pire, une erreur. À l'inverse, choisir systématiquement la configuration la plus légère sacrifie la qualité sur les questions difficiles.

Le problème est donc un problème d'*allocation* : il s'agit de donner à chaque question les ressources dont elle a besoin, ni plus ni moins. Il est délicat pour trois raisons. D'abord, il faut décider *avant* de connaître la réponse, à partir d'indices imparfaits. Ensuite, ces indices ont eux-mêmes un coût : un signal de décision qui demande une génération supplémentaire peut annuler l'économie qu'il est censé produire. Enfin, l'objectif est multiple. On veut réduire la latence et l'énergie, mais sans perdre en qualité, sans augmenter les erreurs et sans sacrifier une catégorie de questions au profit d'une autre. Nous formulons ainsi la problématique de ce travail :

> *Comment allouer automatiquement, et de manière mesurable, les ressources de calcul et de recherche documentaire à chaque requête, afin de réduire le coût d'un système de questions-réponses local sans dégrader significativement la qualité, la factualité et la fiabilité de ses réponses ?*

1.3 Questions de recherche

Cette problématique se décline en quatre questions, qui structurent la suite de l'article.

QR1 Peut-on, à partir d'un signal peu coûteux, choisir le modèle et le niveau de recherche documentaire sans perte de qualité ?

QR2 Peut-on décider de manière fiable quand s'abstenir, en combinant l'incertitude du modèle et la qualité des preuves retrouvées ?

QR3 Quel compromis entre qualité, latence et énergie obtient-on réellement sur un ordinateur personnel, une fois compté le coût des signaux de décision ?

QR4 Les gains sont-ils homogènes selon le type de question, ou certains profils sont-ils sacrifiés ?

1.4 Démarche et contributions

Nous avons abordé ces questions par étapes, en nous imposant une discipline expérimentale stricte. Une revue de la littérature a d'abord permis de situer le problème et de formuler des hypothèses. Nous avons ensuite construit un benchmark et un protocole, puis écrit nos critères de succès *avant* la moindre expérience. Une étude pilote a validé la chaîne de mesure. Des méthodes de référence, fixes ou inspirées de travaux publiés, ont fixé la barre. Nous avons alors conçu et réglé une politique adaptative sur un jeu de développement, avant de la figer et de l'évaluer une seule fois sur un jeu de test jamais consulté. Ce travail apporte cinq contributions :

1. une synthèse de la littérature qui montre que le choix du modèle, la décision de recherche et l'abstention sont presque toujours traités séparément, avec un coût rarement mesuré sur matériel local ;

2. un benchmark et un protocole pré-enregistrés, avec mesure de la latence sur trois passes et de l'énergie par `powermetrics` ;
3. un ensemble de méthodes de référence : six stratégies fixes et deux routeurs inspirés de RouteLLM et d'Adaptive-RAG ;
4. une politique adaptative conjointe, simple et interprétable, fondée sur des signaux quasi gratuits ;
5. une évaluation finale dont nous rapportons honnêtement le résultat, partiellement négatif, avec l'analyse de ses causes et des pistes d'amélioration concrètes.

Organisation de l'article. La section 2 définit les notions employées. La section 3 présente l'état de l'art et en déduit nos hypothèses. La section 4 décrit le benchmark et le protocole, la section 5 l'étude pilote, la section 6 les méthodes de référence et la section 7 la politique adaptative. La section 8 rapporte l'évaluation finale. Nous discutons ces résultats en section 9, examinons les faiblesses de l'étude en section 10, et concluons en section 11.

2 Cadre conceptuel

Plusieurs termes reviennent tout au long de cet article. Nous les précisons ici, car leur usage varie d'un travail à l'autre.

On appelle **routage** le fait de choisir, avant toute génération, le modèle qui traitera une requête parmi plusieurs candidats de tailles ou de coûts différents. La **cascade** en est une variante séquentielle : on interroge d'abord un modèle peu coûteux, puis on passe la main à un modèle plus puissant si la première réponse paraît peu fiable. Le coût du premier modèle est alors payé dans tous les cas, ce qui pèse lourd lorsque l'escalade est fréquente.

La **génération augmentée par récupération** (RAG) consiste à ajouter au prompt des passages extraits d'une base documentaire. Le RAG est dit **adaptatif** lorsqu'il décide, requête par requête, s'il faut chercher, quand chercher et combien de passages retenir.

L'**incertitude** désigne une estimation de la probabilité qu'une réponse soit fausse. Elle peut se lire dans les probabilités internes du modèle, dans la cohérence entre plusieurs réponses échantillonnées, ou dans la confiance que le modèle exprime lui-même. Les **preuves** sont les passages retrouvés ; leur **suffisance** mesure dans quelle mesure ils semblent contenir de quoi répondre.

L'**abstention** est le refus de répondre lorsque le risque d'erreur est jugé trop élevé. On l'évalue par deux grandeurs complémentaires (Kamath et al., 2020) : la **couverture**, part des questions auxquelles le système répond, et le **risque**, part des réponses fausses parmi celles qu'il donne. S'abstenir davantage fait baisser la couverture ; c'est utile si le risque baisse en proportion.

Enfin, le **coût** n'a pas le même sens partout. Selon les travaux, il s'agit du prix facturé par une API, du nombre d'appels au grand modèle, du nombre de tokens, de la latence, de la mémoire ou de l'énergie. Ces mesures ne sont pas interchangeables. Dans ce travail, le coût est celui que subit l'utilisateur d'une machine personnelle : la latence de bout en bout et l'énergie consommée par la puce, complétées par le nombre de tokens.

3 État de l'art

3.1 Méthode de revue

Nous avons interrogé arXiv, ACL Anthology, OpenReview, Google Scholar et Semantic Scholar, ainsi que les actes de NeurIPS, ICLR, AAAI et MLSys. Les requêtes croisaient les notions du sujet et leurs synonymes anglais : *LLM routing*, *model cascade*, *adaptive retrieval*, *adaptive RAG*, *uncertainty estimation*, *selective prediction*, *abstention* et *energy-aware inference*. Nous avons retenu les travaux qui adaptent, requête par requête, le choix du modèle, la recherche documentaire

ou la décision de répondre, à condition qu'ils s'accompagnent d'une évaluation expérimentale, ainsi que les études de coût d'inférence et les jeux de données utiles à notre protocole. Les travaux d'infrastructure, les articles de position sans expérience et les systèmes multi-agents hors du cadre des questions-réponses ont été écartés. Partis de dix-sept références, nous en avons identifié cinquante et une en suivant les citations dans les deux sens et en menant une recherche ciblée sur les travaux de 2025 et 2026 ; quarante-six ont été retenues et classées par thème.

3.2 Routage et cascades de modèles

L'idée de réserver le grand modèle aux requêtes qui le justifient est ancienne, mais elle a pris de l'ampleur avec la multiplication des API payantes. Chen et al. (2023) en donnent une formulation générale avec FrugalGPT, qui enchaîne des modèles de coût croissant et s'arrête dès qu'un évaluateur appris juge la réponse acceptable ; ils rapportent des économies considérables à qualité égale. AutoMix (Aggarwal et al., 2023) remplace cet évaluateur par une auto-vérification du petit modèle, dont le bruit est absorbé par un processus de décision markovien. Yue et al. (2023) utilisent un autre indice : lorsque plusieurs réponses du petit modèle concordent, il y a de bonnes chances qu'il ait raison.

Une seconde famille décide *avant* toute génération. Hybrid LLM (Ding et al., 2024) prédit la difficulté d'une requête pour l'envoyer au petit ou au grand modèle, avec un seuil réglable au moment de l'inférence. RouteLLM (Ong et al., 2024) apprend ce choix à partir de préférences humaines et montre que les routeurs obtenus se transfèrent à d'autres paires de modèles. D'autres travaux recourent à un bandit manchot (Nguyen et al., 2024) ou à un modèle de récompense (Lu et al., 2023). Pour comparer ces approches, RouterBench (Hu et al., 2024) propose un cadre d'évaluation commun.

Les travaux les plus récents affinent la notion de coût et de garantie. Fu et al. (2026) montrent que le coût d'un appel isolé sous-estime largement celui d'un processus qui relance les requêtes, et proposent un signal de difficulté calculé localement avant exécution. RACER (Hao et al., 2026) produit des ensembles de modèles avec un seuil calibré qui autorise l'abstention et contrôle le risque d'erreur de routage, avec des garanties qui ne supposent rien de la distribution des données. La revue de Varangot-Reille et al. (2026) dresse un bilan du domaine et en souligne deux faiblesses : des protocoles expérimentaux peu standardisés et une prise en compte quasi exclusive du coût financier.

3.3 RAG adaptatif

Que la recherche documentaire ne soit pas toujours utile, Mallen et al. (2023) l'ont montré de façon convaincante : les modèles retiennent bien les connaissances populaires et mal les connaissances rares, si bien que récupérer seulement lorsque c'est nécessaire améliore à la fois la précision et le coût. Les méthodes qui ont suivi diffèrent surtout par l'indice qui déclenche la recherche. SKR (Wang et al., 2023) s'appuie sur ce que le modèle sait de lui-même ; FLARE (Jiang et al., 2023) et DRAGIN (Su et al., 2024) déclenchent la recherche en pleine génération, dès que le modèle hésite ; Self-RAG (Asai et al., 2024) apprend au modèle à émettre des jetons de réflexion qui jugent la pertinence des passages, au prix d'un entraînement dédié.

Adaptive-RAG (Jeong et al., 2024) est la référence la plus proche de notre problème. Un petit classifieur y prédit la complexité de la question et choisit entre aucune recherche, une recherche en une étape et une recherche itérative. HyPA-RAG (Kalra et al., 2024) transpose cette idée aux paramètres de recherche dans le domaine juridique. D'autres travaux jugent plutôt les documents que la question : Corrective RAG (Yan et al., 2024) évalue les passages retrouvés et décide de les utiliser, de les compléter ou de les écarter, et Tiny-Critic RAG (Wu et al., 2026) confie ce contrôle à un petit modèle pour en réduire la latence.

Une étude comparative de Moskvoretskii et al. (2025) a particulièrement orienté notre travail : sur une trentaine de méthodes, des estimateurs d'incertitude simples rivalisent avec des pipelines bien plus complexes, et les dépassent souvent en efficacité. Plus récemment, Self-Routing RAG

(Wu et al., 2025) traite le modèle lui-même comme une source de connaissance parmi d'autres, RAGRouter-Bench (Wang et al., 2026) formule le choix d'un paradigme de RAG comme un problème de routage, et Pre-Route (Chen et al., 2026b) arbitre entre RAG et contexte long. Enfin, plusieurs travaux réduisent le coût du contexte lui-même, en ne conservant que les phrases utiles (Arefeen et al., 2024; Akesson and Santos, 2024; Chen et al., 2026a).

3.4 Incertitude et abstention

Pour décider de s'abstenir, encore faut-il savoir quand le modèle risque de se tromper. Kadavath et al. (2022) montrent que les grands modèles sont souvent bien calibrés lorsqu'on leur demande de juger leurs propres réponses. L'entropie sémantique (Kuhn et al., 2023) et SelfCheckGPT (Manakul et al., 2023) comparent plusieurs réponses échantillonnées : leur désaccord trahit une incertitude ou une hallucination. Ces signaux sont efficaces, mais ils exigent plusieurs générations par question, ce qui les rend coûteux précisément dans le cadre qui nous occupe. La confiance exprimée en mots par le modèle n'est, quant à elle, fiable qu'après un apprentissage spécifique (Lin et al., 2022).

Kamath et al. (2020) ont posé les bases de la réponse sélective, avec les notions de couverture et de risque que nous reprenons, et montré qu'un calibrateur appris fait mieux que les probabilités brutes, trop confiantes hors du domaine d'entraînement. La revue de Wen et al. (2024) constate que l'abstention reste rarement intégrée aux systèmes de RAG comme une décision à part entière.

3.5 Coût et énergie de l'inférence

Samsi et al. (2023) ont mesuré l'énergie de l'inférence de LLaMA selon la taille du modèle sur des GPU de centre de données. Luccioni et al. (2022) ont estimé l'empreinte carbone de BLOOM sur tout son cycle de vie, et Luccioni et al. (2024) ont comparé le coût énergétique de nombreux modèles et tâches, montrant que les modèles génératifs généralistes sont bien plus gourmands que les modèles spécialisés. Ces travaux *mesurent* l'énergie, mais aucun ne cherche à la *réduire* par des décisions prises question par question, et tous portent sur du matériel de centre de données.

3.6 Jeux de données et pratiques d'évaluation

Les jeux de données disponibles couvrent bien les profils de questions qui nous intéressent. SQuAD 2.0 (Rajpurkar et al., 2018) fournit des questions à réponse unique et des questions sans réponse ; HotpotQA (Yang et al., 2018) et MuSiQue (Trivedi et al., 2022) des questions qui exigent de combiner plusieurs documents ; le CRAG Benchmark (Yang et al., 2024) des questions variées où les hallucinations sont pénalisées. Sur le plan méthodologique, deux leçons nous ont paru essentielles. D'une part, Li et al. (2026) montrent que des signaux issus de la recherche, pourtant corrélés à l'utilité d'une étape supplémentaire, n'améliorent pas de façon fiable un routeur qui ne lit que la question : tout nouveau signal doit donc être comparé à un tel contrôle. D'autre part, RouterBench et RAGRouter-Bench évaluent qualité et coût sous forme de compromis plutôt qu'avec une métrique unique, ce que nous ferons aussi.

3.7 Synthèse et positionnement

Le tableau 1 situe les travaux les plus proches selon les trois décisions qui nous intéressent.

Travail	Choix du modèle	Décision RAG	Abstention	Signal de décision	Coût évalué	Local
FrugalGPT	✓	✗	✗	Évaluateur appris (après génération)	Prix API	✗
Hybrid LLM	✓	✗	✗	Difficulté prédite (avant)	Appels au grand modèle	∼
RouteLLM	✓	✗	✗	Préférences humaines	Prix, appels	✗
AutoMix	✓	✗	✗	Auto-vérification (après)	Calcul	✗
RACER	✓	✗	✓	Score calibré	Taille d'ensemble	✗
Adaptive-RAG	✗	✓	✗	Complexité de la question	Temps, étapes	✗
Corrective RAG	✗	✓	✗	Qualité des documents	–	✗
Moskvoretskii et al.	✗	✓	✗	Incertitude	Appels, récupérations	✗
RAGRouter-Bench	∼	✓	✗	Adéquation requête–corpus	Ressources	✗
Kamath et al.	✗	✗	✓	Calibrateur appris	–	✗
Ce travail	✓	✓	✓	**Preuve BM25 et confiance, coût compté**	**Latence, tokens, énergie**	✓

TABLE 1 – Positionnement des travaux les plus proches. ✓ : traité ; ✗ : non traité ; ∼ : partiellement.

De cette revue se dégagent quatre manques. Les trois décisions sont presque toujours prises *isolément* : on choisit le modèle sans se soucier de la recherche, ou l'inverse, et l'abstention est étudiée à part. Le coût est surtout *financier*, rarement mesuré sur la machine de l'utilisateur. Le coût des signaux eux-mêmes, qu'il s'agisse de plusieurs générations ou d'un modèle juge, est rarement retranché du gain annoncé. Enfin, l'évaluation est parfois optimiste : un contrôle « question seule » fait souvent défaut, et une moyenne globale peut masquer des pertes sur une catégorie de questions. Le tableau 2 précise ce qui distingue notre travail des plus proches.

Travail	Ce qu'il fait déjà	Ce qui reste ouvert
Adaptive-RAG	Complexité de la question → stratégie de recherche	Choix du modèle, abstention, mesure locale
Moskvoretskii et al.	Signaux d'incertitude simples pour décider de la recherche	Plusieurs tailles de modèle, abstention, énergie
RAGRouter-Bench	Routage entre paradigmes de RAG avec coût en ressources	Abstention, matériel local, décision conjointe
Fu et al. (2026)	Signal local avant exécution, coût réel	Recherche adaptative, abstention
RACER	Routage avec abstention calibrée	Recherche documentaire, coût local

TABLE 2 – Ce qui distingue ce travail des plus proches.

Ces constats nous ont conduit à formuler quatre hypothèses de travail, qui répondent aux questions de recherche de l'introduction.

H1 Un signal peu coûteux, calculé avant génération, permet d'orienter une part significative des requêtes vers le petit modèle sans perte de qualité au-delà de la marge fixée.

H2 Combiner l'incertitude du modèle et la suffisance des preuves améliore le compromis entre couverture et risque par rapport à chacun de ces signaux pris isolément.

H3 Une politique conjointe domine les stratégies fixes sur la courbe qualité–coût, coût de ses propres signaux compris.

H4 Les gains varient selon la catégorie de question, et doivent donc être rapportés par catégorie.

4 Benchmark et protocole

4.1 Construction du benchmark

Nous avons construit un benchmark à partir de deux jeux publics, tous deux sous licence CC BY-SA 4.0, dont l'empreinte SHA-256 est vérifiée à chaque construction. Le jeu de développement de SQuAD 2.0 (Rajpurkar et al., 2018) couvre trente-cinq articles de Wikipédia et mêle des questions à réponse et des questions sans réponse, rédigées pour ressembler aux premières. Le jeu de développement de HotpotQA en configuration *distractor* (Yang et al., 2018) contient des questions qui exigent de combiner deux pages Wikipédia.

Le corpus documentaire est la bibliothèque dans laquelle puise la recherche ; il ne sert pas à entraîner les modèles. Il réunit les 1 204 paragraphes de SQuAD 2.0 et les 2 089 paragraphes associés aux questions HotpotQA retenues, soit 3 293 passages. Toutes les questions interrogent ce même corpus, si bien que la recherche doit retrouver les bons passages parmi plusieurs milliers.

Les questions se répartissent en trois catégories équilibrées (tableau 3). Les questions *simples* trouvent leur réponse dans un seul passage ; les questions *complexes* demandent d'en combiner deux ; les questions *non couvertes* n'ont pas de réponse dans le paragraphe d'origine, et le comportement attendu est alors l'abstention. Pour éviter toute fuite entre réglage et évaluation, les questions SQuAD sont séparées *par article* : un paragraphe ne sert jamais à la fois au développement et au test. Nous avions prévu une quatrième catégorie, les questions ambiguës, qui devaient être rédigées à la main ; elle n'a pas pu être constituée à temps.

Catégorie	Source	Définition	Attendu	Dév.	Test
Simple	SQuAD 2.0	La réponse se trouve dans un seul passage	Répondre	60	150
Complexe	HotpotQA	La réponse exige de combiner deux passages	Répondre	60	150
Non couverte	SQuAD 2.0	Le passage d'origine ne contient pas la réponse	S'abstenir	60	150
Total				180	450

TABLE 3 – Catégories du benchmark et répartition entre développement et test (graine 2026).

4.2 Environnement et génération

Toutes les expériences ont été menées sur un Apple M3 Pro doté de 12 cœurs et de 18 Go de mémoire unifiée, avec Python 3.12 et la bibliothèque MLX. Nous utilisons deux modèles quantifiés en 4 bits et préchargés en mémoire : Qwen2.5-1.5B-Instruct, que nous appelons le *petit* modèle, et Qwen3-4B-Instruct-2507, le *grand*. Les révisions exactes des modèles sont consignées dans le dépôt. Le décodage est glouton et limité à 32 tokens, de sorte que les sorties sont entièrement déterministes. Les prompts sont identiques pour tous les systèmes et contiennent une consigne d'abstention explicite : répondre exactement `UNANSWERABLE` lorsque la réponse est inconnue ou absente des passages. Trois requêtes de chauffe précèdent chaque série.

4.3 Mesures

La métrique principale est un *score global* calculé pour chaque question. Lorsqu'une réponse est attendue, il vaut le F1 lexical entre la réponse produite et la meilleure réponse de référence. Pour une question non couverte, il vaut 1 si le système s'abstient et 0 sinon. Une réponse est jugée correcte lorsque son F1 atteint 0,5, ce qui permet de calculer la couverture et le risque. La latence est mesurée de bout en bout, recherche et signaux compris, avec une horloge monotone ; nous en rapportons la moyenne et le 95e centile. L'énergie est mesurée par `powermetrics`, qui relève toutes les 200 ms la puissance du processeur, du GPU et du moteur neuronal ; elle est intégrée sur la durée de chaque série.

Les intervalles de confiance à 95 % sont obtenus par bootstrap sur 10 000 rééchantillonnages des questions. Comme tous les systèmes répondent aux mêmes questions, les écarts entre systèmes sont calculés question par question et rééchantillonnés ensemble. Les résultats par catégorie sont systématiquement rapportés à côté de la moyenne.

4.4 Critères de succès pré-enregistrés

Nous avons écrit et publié les critères suivants avant toute expérience, pour nous interdire de redéfinir le succès après coup.

Critère principal. Sur le jeu de test et par rapport à la stratégie de référence B (grand modèle et trois passages), la latence moyenne doit baisser d'au moins 20 % et le score global ne doit pas perdre plus de 2 points. La non-infériorité n'est revendiquée que si la borne supérieure de l'intervalle de confiance à 95 % de la perte reste sous 2 points.

Critères secondaires. Le risque doit être inférieur à celui de B ; la méthode doit faire mieux qu'un routeur qui ne lit que la question ; aucune catégorie ne doit perdre plus de 5 points.

Résultat négatif. Si le critère principal n'est pas atteint, le résultat est rapporté tel quel, avec l'analyse de ses causes.

Tout réglage se fait sur le jeu de développement. Le protocole a été figé dans une configuration versionnée (étiquette Git `protocole-v1.0`), puis la politique adaptative (étiquette `politique-v1.0`), avant que le jeu de test ne soit lu. Celui-ci n'a été exécuté qu'une fois.

5 Étude pilote

Avant d'investir dans une méthode adaptative, nous avons voulu vérifier que le problème en valait la peine. Une étude pilote a donc exécuté quatre stratégies fixes sur le jeu de développement, avec une recherche dense fondée sur des plongements MiniLM multilingues.

Stratégie	Score	Couv.	Risque	Latence	Tokens	Énergie / q.
A – grand modèle, sans RAG	0,313	0,31	0,82	0,25 s	62	3,8 J
B – grand modèle avec RAG	**0,424**	0,56	0,53	0,99 s	579	22,8 J
C – petit modèle, sans RAG	0,233	0,46	0,93	0,13 s	84	1,9 J
D – petit modèle avec RAG	0,298	0,78	0,71	0,39 s	601	8,6 J
Oracle	*0,623*	–	–	*0,30 s*	–	–

TABLE 4 – Étude pilote sur le jeu de développement (180 questions, recherche dense). L'oracle choisit, pour chaque question, la stratégie la moins coûteuse qui obtient le meilleur score.

Le pilote a d'abord confirmé que l'enjeu est réel. La stratégie B est près de huit fois plus lente que la stratégie C et consomme douze fois plus d'énergie ; le RAG augmente à la fois la durée du calcul et la puissance appelée, qui passe d'environ 17 à 24 W. Il a montré ensuite qu'aucune stratégie fixe ne domine partout : B l'emporte sur les questions simples, D sur les complexes, A sur les non couvertes. L'oracle, qui choisirait la bonne stratégie pour chaque question, atteindrait 0,62 en 0,30 s. Cette borne est inaccessible en pratique, mais elle mesure la marge disponible.

Deux observations ont orienté la suite. L'abstention laissée au seul prompt est mal calibrée : le petit modèle avec RAG répond presque toujours, avec un risque de 0,71, tandis que le grand modèle sans RAG s'abstient sur 69 % des questions. La décision de s'abstenir doit donc relever de la politique, et non du seul modèle. Par ailleurs, la recherche dense est le maillon faible pour les questions complexes : les deux passages utiles ne sont retrouvés ensemble que dans 22 % des cas.

6 Méthodes de référence

6.1 Choix de la recherche documentaire

Le pilote nous a incité à reconsidérer la recherche. Nous avons comparé sur le jeu de développement la recherche dense, une recherche lexicale BM25 ($k_1 = 1{,}5$, $b = 0{,}75$) et une recherche hybride qui fusionne les deux classements par la somme des inverses des rangs.

	Simple			Complexe			
Méthode	@1	@3	@6	@1	@3	@6	**Temps / requête**
Dense	0,48	0,60	0,70	0,00	0,22	0,35	7,4 ms
BM25	**0,65**	**0,82**	**0,87**	0,00	**0,37**	**0,50**	**0,2 ms**
Hybride	0,60	0,72	0,80	0,00	0,33	0,48	7,1 ms

TABLE 5 – Rappel de la recherche sur le jeu de développement. Pour une question complexe, les deux passages utiles doivent être retrouvés, si bien que le rappel@1 est nul par construction.

BM25 l'emporte nettement sur les questions qui ont une réponse, à chaque profondeur, tout en étant quarante fois plus rapide. La recherche hybride retrouve un peu plus souvent le paragraphe d'origine des questions non couvertes, mais ce critère compte peu pour elles, puisque le comportement attendu est de s'abstenir. Ce résultat s'explique par la nature des questions, riches en noms propres repris du texte source, et par la modestie du modèle de plongements. Nous avons donc retenu BM25 et figé ce choix dans le protocole.

6.2 Six stratégies fixes

Nous avons complété les quatre stratégies du pilote par deux stratégies à six passages, afin de disposer d'un espace de décision assez riche.

Stratégie	**Score**	**IC 95 %**	**Simple**	**Complexe**	**Non couv.**	**Couv.**	**Risque**	**Latence**
A – grand, 0 passage	0,312	[0,248 ; 0,378]	0,07	0,09	0,78	0,31	0,82	0,25 s
B – grand, 3 passages	**0,541**	[0,472 ; 0,610]	0,68	0,36	0,58	0,62	0,39	0,92 s
C – petit, 0 passage	0,233	[0,174 ; 0,294]	0,05	0,07	0,58	0,46	0,93	0,13 s
D – petit, 3 passages	0,330	[0,267 ; 0,397]	0,48	0,29	0,22	0,83	0,69	0,37 s
E – grand, 6 passages	0,570	[0,501 ; 0,637]	0,70	0,45	0,57	0,67	0,40	1,63 s
F – petit, 6 passages	0,372	[0,308 ; 0,439]	0,53	0,37	0,22	0,89	0,66	0,62 s

TABLE 6 – Stratégies fixes avec BM25 sur le jeu de développement (180 questions). Les colonnes Simple, Complexe et Non couv. donnent le score par catégorie.

Avec BM25, la référence B passe de 0,424 à 0,541 : la barre que devra franchir la méthode adaptative s'est donc relevée. La stratégie E améliore B de 2,9 points, surtout sur les questions complexes, mais au prix de 76 % de latence supplémentaire, et l'écart n'est pas significatif. Une recherche plus large n'est utile que pour une partie des questions, ce qui plaide pour une décision au cas par cas.

6.3 Routeurs de référence

Pour qu'un éventuel succès de notre méthode ait un sens, il fallait la confronter à des routeurs issus de la littérature. Nous en avons construit deux, qui ne lisent que le texte de la question, représenté par un TF-IDF sur les mots et les paires de mots complété de quelques indicateurs simples (longueur, noms propres, chiffres, mot interrogatif). La génération étant déterministe, la réponse d'un routeur est celle de la stratégie qu'il choisit ; son temps de calcul est ajouté à la latence.

Le premier, **R1**, reprend l'esprit de RouteLLM et de Hybrid LLM et sert de contrôle « question seule » au sens de Li et al. (2026). Une régression ridge prédit, pour chaque stratégie s, le score $\hat{y}_s(q)$ qu'elle obtiendrait sur la question q, et le routeur retient $\arg\max_s \hat{y}_s(q) - \lambda \bar{t}_s$, où $\bar{t}_s$ est la latence moyenne de s. Nous retenons le meilleur réglage au moins 20 % plus rapide que B ($\lambda = 0{,}113$). Le second, **R2**, est une version simplifiée d'Adaptive-RAG (Jeong et al., 2024) : une régression logistique prédit la complexité de la question et choisit, avec le grand modèle, entre aucune recherche, trois passages et six passages.

Méthode	**Score**	**Écart à B**	**IC 95 % de l'écart**	**Latence**	**Couv.**	**Risque**
B – référence	0,541	–	–	0,92 s	0,62	0,39
R1 – question seule	0,425	−11,6	[−18,5; −4,8]	0,74 s	0,58	0,57
R2 – Adaptive-RAG simplifié	0,522	−1,9	[−7,4; +3,3]	0,85 s	0,49	0,39
Oracle (A à F)	*0,762*	+22,1	[+16,5; +27,8]	*0,40 s*	0,58	0,19

TABLE 7 – Routeurs de référence, validation croisée en cinq plis sur le jeu de développement.

Le verdict est sans appel pour R1 : contraint d'être 20 % plus rapide que B, il perd près de douze points. Sur ce benchmark, la formulation de la question ne permet pas de prédire quelle stratégie réussira. R2 conserve la qualité de B mais ne gagne que 8 % de latence, et ne choisit jamais le petit modèle. Pendant ce temps, l'oracle montre qu'on pourrait gagner vingt-deux points tout en divisant la latence par plus de deux. La conclusion s'impose : une méthode adaptative doit regarder autre chose que la question, à commencer par ce que la recherche a trouvé.

7 Une politique adaptative fondée sur la preuve

7.1 Deux signaux presque gratuits

Le premier signal mesure la *suffisance des preuves*. La recherche BM25 renvoie, pour une question q, le score S_1 du meilleur passage. Ce score brut dépend de la longueur et de la rareté des mots de la question ; nous le rapportons donc au score maximal qu'un passage pourrait atteindre :

$$\text{preuve}(q) \;=\; \frac{S_1}{\sum_{t \in q} \text{idf}(t)\,(k_1 + 1)} \;\in\; [0, 1].$$

Une valeur proche de 1 indique qu'un passage contient tous les mots importants de la question. Ce signal est un sous-produit de la recherche et ne coûte que 0,2 ms.

Le second signal est la *confiance du modèle*. Pendant le décodage, nous relevons la probabilité de chaque token produit et en retenons soit la moyenne géométrique, soit le minimum. Le décodage n'est pas modifié : nous avons vérifié que les réponses restent identiques à celles des stratégies fixes. Le tableau 8 indique ce que valent ces signaux pour distinguer les bonnes réponses des mauvaises.

Signal	**Ce qu'on cherche à prédire**	**AUC**
Confiance du petit modèle, 3 passages	Sa réponse est correcte	0,74
Confiance du grand modèle, 3 passages	Sa réponse est correcte	0,63
Confiance du grand modèle, 6 passages	Sa réponse est correcte	0,70
Preuve BM25	La réponse de B est correcte	0,55
Preuve BM25 faible	La question est non couverte	0,62

TABLE 8 – Pouvoir prédictif des signaux sur le jeu de développement (AUC de 0,5 pour le hasard, de 1 pour une prédiction parfaite).

Ces signaux sont informatifs, sans plus : aucun ne dépasse 0,74. Il faudra donc les combiner avec prudence.

7.2 La politique

La politique traite chaque question en quelques étapes. Elle lance d'abord une recherche BM25 de six passages et calcule la preuve. Si celle-ci est inférieure à un seuil τ_0, elle s'abstient immédiatement, sans aucune génération : c'est l'*abstention précoce*. Sinon, elle peut, en option, interroger le petit modèle avec trois passages et retenir sa réponse si sa confiance dépasse un seuil τ_p ; c'est la *cascade*. À défaut, elle interroge le grand modèle, avec six passages si la preuve est inférieure à τ_k et trois sinon. Elle s'abstient enfin si le grand modèle avoue son ignorance ou si sa confiance tombe sous τ_a : c'est l'*abstention finale*. Le coût de chaque étape effectivement exécutée, signaux compris, est compté dans la latence.

7.3 Réglage et conception

Comme les sorties sont déterministes, la réponse de la politique à une question est celle de l'étape où elle s'arrête. Il suffit donc de calculer une fois les sorties de chaque étape pour simuler exactement la politique, latences comprises, sous n'importe quel réglage. Nous avons ainsi exploré 246 510 réglages (type de confiance, seuils, cascade ou non) et retenu celui qui maximise le score global sous la contrainte d'une latence moyenne au plus égale à 78 % de celle de B. Les deux points de marge par rapport au critère de 80 % devaient protéger contre la variabilité d'un jeu de questions à l'autre. Comme le réglage est choisi sur les questions mêmes qui servent à l'évaluer, son score est optimiste ; nous l'avons donc aussi estimé par validation croisée en cinq plis.

La politique n'est pas sortie telle quelle de nos premières réflexions. Notre première conception était une cascade pure : petit modèle d'abord, grand modèle en cas de doute, abstention en dernier recours. Sous la contrainte de latence, elle n'atteignait que 0,395, soit près de quinze points de moins que B. Le petit modèle se trompe trop souvent, sa confiance ne le révèle qu'en partie, et chaque escalade paie deux générations. C'est en ajoutant l'abstention précoce sur preuve faible que la politique est devenue compétitive ; le réglage, qui pouvait toujours activer la cascade, a alors choisi de s'en passer. Nous rapportons ce cheminement par transparence : il montre que la politique résulte d'une exploration sur le jeu de développement, ce qui rend l'évaluation finale d'autant plus nécessaire.

Le réglage figé est le suivant : abstention précoce si la preuve est inférieure à 0,265 ; six passages si elle est inférieure à 0,286 ; abstention finale si la probabilité d'au moins un token de la réponse tombe sous 0,40 ; pas de cascade. En langage courant, la politique se lit ainsi : lorsque les documents ne contiennent manifestement pas les mots de la question, ne rien générer et s'abstenir ; sinon, interroger le grand modèle, en lui donnant davantage de passages si la preuve est limite, et s'abstenir s'il hésite.

7.4 Résultats sur le jeu de développement

Méthode	Score	Écart à B	Latence (gain)	Couv.	Risque
B – référence	0,541	–	0,92 s	0,62	0,39
R1 – question seule	0,425	−11,6	0,74 s (20 %)	0,58	0,57
R2 – Adaptive-RAG simplifié	0,522	−1,9	0,85 s (8 %)	0,49	0,39
Politique (réglage figé)	**0,552**	**+1,1**	**0,71 s (23 %)**	0,48	**0,34**
Politique (validation croisée)	0,520	−2,1	0,72 s (22 %)	0,44	0,35

TABLE 9 – Politique adaptative sur le jeu de développement (180 questions).

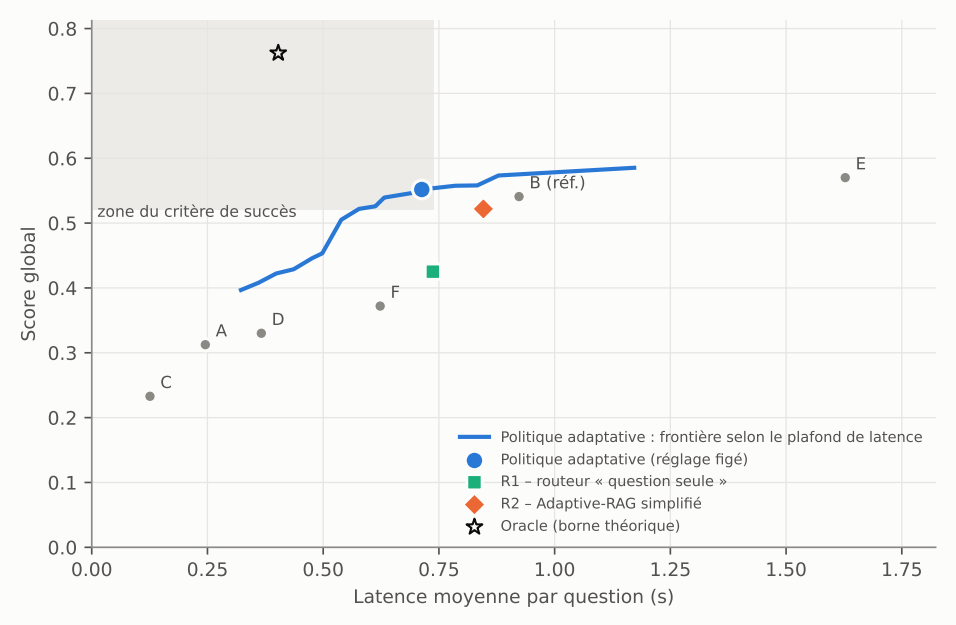


FIGURE 1 – Compromis qualité–latence sur le jeu de développement. La courbe bleue indique, pour chaque plafond de latence, le meilleur score que peut atteindre la politique ; le point bleu correspond au réglage figé. La zone grisée est celle du critère de succès.

Sur le jeu de développement, la politique remplissait le critère principal, avec un léger gain de score et 23 % de latence en moins, et elle devançait les deux routeurs. Deux signaux d'alerte étaient néanmoins visibles. La validation croisée estimait une perte de 2,1 points, juste à la limite du critère. Surtout, les questions simples perdaient quatorze points, en échange d'un gain de vingt-deux points sur les questions non couvertes : le critère de robustesse n'était déjà pas satisfait.

8 Évaluation finale

8.1 Conditions d'exécution

Une fois la politique figée et publiée, nous avons exécuté les six stratégies fixes et la politique sur les 450 questions du jeu de test, en trois passes complètes. Dans chaque passe, l'ordre des méthodes était tiré au hasard, afin de répartir l'effet de l'échauffement de la machine ; la latence retenue pour chaque question est la médiane des trois passes. La politique a été exécutée réellement, de bout en bout. Les routeurs R1 et R2, entraînés sur l'ensemble du jeu de développement, ont été appliqués aux sorties du test. L'énergie a été mesurée pendant toute l'évaluation, soit plus de 30 000 relevés, pour une puissance au repos de 2,7 W.

Deux contrôles attestent la solidité de l'exécution. Les réponses sont identiques d'une passe à l'autre pour toutes les méthodes. Et la politique exécutée reproduit exactement sa simulation : même score sur chacune des questions, même latence moyenne à un millième de seconde près.

8.2 Résultats

Méthode	Score	IC 95 %	Écart à B	Latence	P95	Couv.	Risque
A – grand, sans RAG	0,310	[0,269 ; 0,351]	−22,7	0,27 s	0,52 s	0,45	0,76
B – grand, 3 passages (réf.)	**0,537**	[0,494 ; 0,581]	–	**0,87 s**	1,26 s	0,73	0,42
C – petit, sans RAG	0,223	[0,187 ; 0,261]	−31,4	0,13 s	0,25 s	0,51	0,93
D – petit, 3 passages	0,414	[0,372 ; 0,457]	−12,3	0,35 s	0,53 s	0,88	0,57
E – grand, 6 passages	0,567	[0,523 ; 0,610]	+3,0	1,57 s	2,23 s	0,75	0,40
F – petit, 6 passages	0,398	[0,357 ; 0,440]	−13,9	0,61 s	0,87 s	0,94	0,60
R1 – question seule	0,489	[0,445 ; 0,534]	−4,7	0,74 s	1,61 s	0,68	0,48
R2 – Adaptive-RAG simplifié	0,517	[0,473 ; 0,561]	−2,0	0,82 s	1,39 s	0,68	0,44
Politique adaptative	**0,550**	[0,505 ; 0,594]	**+1,3**	**0,79 s**	1,38 s	0,64	**0,40**
Oracle	*0,797*	[0,762 ; 0,831]	+26,0	*0,36 s*	1,09 s	0,64	0,18

TABLE 10 – Résultats sur le jeu de test (450 questions). La latence est la moyenne, sur les questions, de la médiane des trois passes.

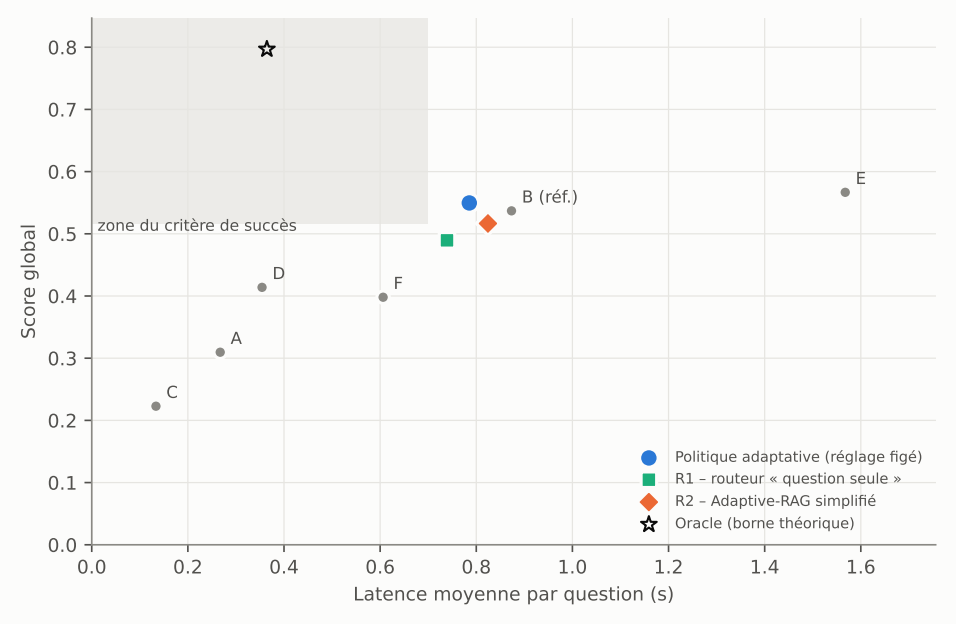


FIGURE 2 – Compromis qualité–latence sur le jeu de test. La politique (en bleu) se situe au-dessus de B, mais à droite de la zone du critère de succès.

La politique obtient le meilleur score de toutes les méthodes réalisables à l'exception de E, qui est deux fois plus lente. Elle devance B d'un peu plus d'un point, fait moins d'erreurs lorsqu'elle répond, et dépasse nettement les deux routeurs. Mais sa latence moyenne n'est inférieure que de 10 % à celle de B.

8.3 Verdict

Critère	Mesure	Atteint
Latence moyenne réduite d'au moins 20 %	−10,1 % (0,786 contre 0,873 s)	✗
Perte de score d'au plus 2 points	gain de 1,3 point	✓
Non-infériorité (borne haute de l'IC de la perte sous 2 points)	[−4,1 ; +1,6]	✓
Critère principal		✗
Risque inférieur à B	0,395 contre 0,421	✓
Meilleur score que R1 et R2	0,550 contre 0,489 et 0,517	✓
Aucune catégorie ne perd plus de 5 points	simple −7,6 ; complexe −3,3 ; non couverte +14,7	✗

Table 11 – Verdict sur les critères pré-enregistrés.

Le verdict est donc nuancé. La qualité est préservée, la non-infériorité est démontrée, la fiabilité s'améliore et les deux routeurs de référence sont battus. Mais l'économie de latence n'atteint que la moitié de l'objectif, et les questions simples paient le prix de l'abstention. L'hypothèse principale, dans la forme où nous l'avions enregistrée, est rejetée.

8.4 Analyse par catégorie

Méthode	Simple	Complexe	Non couverte
B – référence	0,737	0,468	0,407
E – grand, 6 passages	0,730	0,537	0,433
R1	0,562	0,453	0,453
R2	0,635	0,455	0,460
Politique adaptative	0,661 (−7,6)	0,435 (−3,3)	**0,553** (+14,7)
Oracle	0,842	0,682	0,867

Table 12 – Score par catégorie sur le jeu de test (150 questions chacune). Entre parenthèses, l'écart à B en points.

La politique échange de la qualité sur les questions qui ont une réponse contre de la fiabilité sur celles qui n'en ont pas. Sur les 450 questions, elle a déclenché 66 abstentions précoces et fait répondre le grand modèle 359 fois avec trois passages et 25 fois avec six ; 98 de ces réponses ont ensuite été remplacées par une abstention finale. Les abstentions précoces frappent surtout les questions non couvertes, ce qui est le but recherché, mais aussi 18 questions simples et 12 complexes : un score BM25 faible ne signifie pas toujours que la réponse est absente.

8.5 Ablations

	Test		**Développement**	
Variante (chacune réglée sur le développement)	Score	Latence	Score	Latence
Politique complète	**0,550**	0,79 s	0,552	0,71 s
Sans recherche adaptative (toujours 3 passages)	0,547	0,74 s	0,548	0,68 s
Sans abstention finale	0,547	0,79 s	0,546	0,71 s
Sans abstention précoce (cascade petit → grand)	0,464	0,64 s	0,395	0,70 s
Sans aucune abstention	0,457	0,64 s	0,389	0,70 s

Table 13 – Ablations : chaque composante est retirée et la politique est réglée à nouveau.

Les ablations désignent clairement la composante décisive : sans abstention précoce, la politique perd près de neuf points sur le test. La recherche adaptative et l'abstention finale, elles, n'apportent chacune que trois dixièmes de point, ce qui reste dans le bruit statistique. Sur le test, s'en tenir toujours à trois passages aurait même été plus rapide, pour un score équivalent.

8.6 Énergie

Méthode	Énergie par question	Écart à B	Puissance moyenne
C – petit, sans RAG	1,8 J	−90 %	16,4 W
A – grand, sans RAG	3,9 J	−80 %	17,0 W
D – petit, 3 passages	7,3 J	−62 %	23,2 W
F – petit, 6 passages	13,5 J	−30 %	24,9 W
Politique adaptative	**17,5 J**	**−9** %	24,8 W
B – grand, 3 passages (réf.)	19,3 J	–	24,7 W
E – grand, 6 passages	37,7 J	+95 %	26,4 W

TABLE 14 – Énergie de la puce (processeur, GPU et moteur neuronal) par question sur le test, moyenne des trois passes.

La puissance appelée varie peu d'une méthode à l'autre, entre 16 et 26 W. L'énergie suit donc essentiellement la durée du calcul, c'est-à-dire le nombre de tokens traités, et l'économie de la politique (−9 %) reflète sa réduction de latence. On notera au passage que doubler le nombre de passages double presque l'énergie, pour trois points de score.

8.7 Du développement au test

	Développement (180)		Test (450)	
Méthode	Score	Latence	Score	Latence
B – référence	0,541	0,92 s	0,537	0,87 s
Politique adaptative	0,552	0,71 s (−23 %)	0,550	0,79 s (−10 %)
R1	0,425	0,74 s	0,489	0,74 s
R2	0,522	0,85 s	0,517	0,82 s
Oracle	0,762	0,40 s	0,797	0,36 s
Abstentions précoces	45 / 180 (25 %)		66 / 450 (15 %)	

TABLE 15 – Comparaison entre jeu de développement et jeu de test.

C'est en comparant les deux jeux que l'échec s'éclaire. La qualité s'est remarquablement bien généralisée : 0,550 sur le test, mieux même que les 0,520 prévus par la validation croisée. Le gain de latence, en revanche, s'est réduit de moitié. Le seuil d'abstention précoce, réglé sur soixante questions par catégorie, écartait un quart des questions de développement, mais seulement 15 % des questions de test. L'écart représente environ trois erreurs-types, ce qui l'attribue à la variabilité d'un trop petit échantillon de réglage. Moins d'abstentions précoces, c'est davantage d'appels au grand modèle, et donc une latence qui se rapproche de celle de B.

9 Discussion

9.1 Retour sur les hypothèses

La première hypothèse est rejetée. Aucun signal calculé avant génération n'a permis d'envoyer des questions au petit modèle sans perte : R1 perd entre cinq et douze points selon le jeu, et la

politique réglée a écarté le petit modèle. La deuxième n'est pas confirmée. La politique fait moins d'erreurs que B, mais la confiance du modèle n'ajoute presque rien à la preuve : sans abstention finale, le score et le risque sont pratiquement inchangés. La troisième est partiellement confirmée. Aucune autre méthode n'est à la fois plus précise et plus rapide que la politique, qui domine B ; mais son avantage de coût reste modeste. La quatrième, enfin, est pleinement confirmée : sept points perdus sur les questions simples, quinze gagnés sur les questions non couvertes. Une moyenne seule aurait masqué ce compromis.

9.2 Ce que nous retenons

Le premier enseignement est que, dans notre cadre, *ce que la recherche a trouvé renseigne mieux que la question elle-même*. À latence comparable, la politique devance R1 de six points et R2 de trois. Ce résultat contraste avec celui de Li et al. (2026), pour qui les signaux issus de la recherche n'améliorent pas de façon fiable un routeur fondé sur la requête. La différence tient peut-être au rôle que nous donnons à ce signal : nous ne l'utilisons pas pour choisir entre plusieurs façons de répondre, mais pour décider de ne pas répondre du tout.

Le deuxième enseignement est que *l'abstention rend l'adaptation rentable*. Ne rien générer lorsque la preuve est faible économise du calcul et évite des erreurs dans le même mouvement. C'est la seule composante dont l'effet dépasse nettement le bruit. Le troisième concerne le petit modèle, qui n'a pas trouvé sa place : sa confiance ne signale ses erreurs que modérément, et chaque escalade fait payer deux générations. Le dernier est une invitation à poursuivre. L'oracle atteint près de 0,80 en 0,36 s : les bons choix existent, et la difficulté est de les prédire.

9.3 Portée pratique

Pour qui déploie un assistant local, ces résultats suggèrent une règle simple et peu risquée : mesurer la suffisance des preuves, qui ne coûte rien, et s'abstenir lorsque les documents ne contiennent manifestement pas les termes de la question. On conserve ainsi la qualité de la configuration complète, on réduit les réponses fausses, et l'on gagne un peu de temps et d'énergie. Il faut toutefois régler le seuil sur un échantillon représentatif, ou mieux, l'exprimer comme une proportion de questions, faute de quoi le gain de temps devient imprévisible.

10 Points faibles et pistes d'amélioration

Une évaluation honnête doit dire ce qui n'a pas marché et pourquoi. Nous présentons ici les faiblesses de l'étude, de la plus lourde de conséquences à la plus secondaire, en proposant pour chacune une voie d'amélioration. Le tableau 16 les récapitule.

10.1 Faiblesses de la méthode

Des seuils absolus réglés sur un petit échantillon. C'est la cause directe de l'échec du critère principal. Un seuil fixé sur une valeur absolue de la preuve, à partir de soixante questions par catégorie, ne déclenche pas la même proportion d'abstentions sur un autre jeu de questions, et la validation croisée, menée sur le même petit échantillon, ne pouvait pas le révéler. Plusieurs remèdes sont possibles et se combinent : exprimer le seuil comme un quantile ou un budget (« s'abstenir sur les x % de questions aux preuves les plus faibles »), ce qui garantit le taux d'abstention et donc la latence ; le calibrer par une méthode conforme, qui offre des garanties de risque sans hypothèse sur la distribution (Hao et al., 2026) ; régler sur au moins cinq cents questions ; et fonder la marge de sécurité sur la variabilité mesurée par bootstrap plutôt que sur une valeur fixée à l'avance.

Des questions simples sacrifiées. L'abstention précoce écarte aussi des questions qui ont une réponse, parce qu'une reformulation ou un synonyme suffit à faire chuter le score lexical. La couverture passe ainsi de 0,73 à 0,64. Un juge de suffisance plus fin, tel un re-classeur de type *cross-encoder* ou un petit modèle d'inférence, dans l'esprit de Corrective RAG (Yan et al., 2024) et de Tiny-Critic RAG (Wu et al., 2026), devrait réduire ces faux négatifs. Combiner preuves lexicale et dense irait dans le même sens, de même qu'une contrainte explicite par catégorie lors du réglage.

Des signaux de confiance faibles. Aucun signal ne dépasse une AUC de 0,74, et l'abstention finale n'apporte presque rien. La probabilité des tokens d'une réponse de quelques mots renseigne mal sur sa justesse. L'entropie sémantique (Kuhn et al., 2023) ou la cohérence entre quelques réponses échantillonnées seraient plus informatives ; pour en contenir le coût, on pourrait les réserver aux cas limites. Une sonde entraînée sur les états internes du modèle (Kadavath et al., 2022) est une autre piste.

Une recherche adaptative sans effet. Passer de trois à six passages ne concerne que 6 % des questions et n'apporte rien de mesurable. Or les questions complexes n'ont qu'un rappel@3 de 0,37 : le problème n'est pas le nombre de passages, mais la recherche elle-même. Décomposer les questions à plusieurs étapes, déclencher une recherche itérative lorsque le modèle hésite (Jiang et al., 2023; Su et al., 2024) et compresser le contexte pour réduire le coût des passages supplémentaires (Arefeen et al., 2024) seraient plus prometteurs.

Un petit modèle inutilisé. La politique finale ne recourt qu'à un seul modèle, si bien que la dimension « choix du modèle » de notre problématique n'est pas réellement exploitée. Un petit modèle plus performant ou spécialisé dans l'extraction pourrait changer la donne. On peut aussi lui confier un rôle moins exigeant, comme la reformulation de requêtes, ou recourir au décodage spéculatif, où le petit modèle accélère le grand sans modifier ses réponses.

Une latence de queue dégradée et un écart persistant à l'oracle. Le 95e centile de la latence de la politique (1,38 s) dépasse celui de B (1,26 s), à cause des réponses à six passages ; il faudrait l'intégrer à l'objectif de réglage. Quant à l'écart de vingt-cinq points avec l'oracle, seul un routeur appris combinant question, preuve et confiance, entraîné sur bien plus de données, pourrait espérer le réduire sensiblement.

10.2 Faiblesses du protocole

Un benchmark construit et restreint. Les questions sont en anglais, tirées de deux jeux académiques, et interrogent un corpus fermé de 3 293 passages. Les questions non couvertes sont des questions adverses de SQuAD 2.0, rédigées pour ressembler aux passages, ce qui ne reflète pas l'usage réel. La catégorie des questions ambiguës n'a pas été constituée. Ajouter MuSiQue (Trivedi et al., 2022) et le CRAG Benchmark (Yang et al., 2024), un corpus réel en français et une cinquantaine de questions ambiguës annotées à la main élargirait considérablement la portée des conclusions.

Une métrique lexicale. Le F1 pénalise les réponses justes formulées autrement, qu'il s'agisse d'une paraphrase, d'un synonyme ou d'une date écrite différemment. Un juge sémantique, modèle d'inférence ou LLM évaluateur, validé sur un échantillon annoté à la main, donnerait une image plus fidèle.

Une puissance statistique limitée. Avec 450 questions, l'intervalle de confiance du score s'étend sur environ quatre points et demi de part et d'autre de l'estimation : un écart de deux

points, le seuil même de notre critère, n'est pas détectable avec certitude. Un jeu de test d'au moins 1 500 questions serait nécessaire.

Un seul matériel, une seule mesure. Toutes les mesures proviennent d'un même appareil, avec un seul moteur d'inférence et une seule quantification, et l'énergie est celle de la puce, non la consommation au mur. Répliquer l'étude sur GPU et avec `llama.cpp`, mesurer au wattmètre et rapporter la mémoire consolideraient les conclusions sur le coût.

Une conception explorée sur le développement. Deux conceptions ont été essayées avant de figer la politique. C'est documenté, mais cela accroît le risque de sur-ajustement au jeu de développement. Il faudrait à l'avenir pré-enregistrer aussi la famille de politiques envisagées, et évaluer toute amélioration sur un jeu de test neuf, celui-ci étant désormais consommé.

Une revue à approfondir. Notre état de l'art s'appuie largement sur les résumés des articles. Une lecture intégrale des travaux les plus proches (Adaptive-RAG, Moskvoretskii et al., 2025, RAGRouter-Bench, RACER et Li et al., 2026) est nécessaire pour confirmer le positionnement proposé au tableau 1.

Faiblesse	Amélioration principale	Priorité
Seuils absolus, petit jeu de réglage	Seuils en quantiles ou budget, calibration conforme, au moins 500 questions	Haute
Perte sur les questions simples	Juge de suffisance léger, contrainte par catégorie	Haute
Puissance statistique limitée	Jeu de test d'au moins 1 500 questions	Haute
Signaux de confiance faibles	Cohérence entre échantillons sur les cas limites	Moyenne
Recherche adaptative sans effet	Décomposition des questions à plusieurs étapes	Moyenne
Benchmark restreint	MuSiQue, CRAG, corpus français, questions ambiguës	Moyenne
Métrique lexicale	Juge sémantique validé	Moyenne
Petit modèle inutilisé	Autre petit modèle, décodage spéculatif	Moyenne
Revue fondée sur les résumés	Lecture intégrale des travaux proches	Moyenne
Latence de queue	95e centile dans l'objectif de réglage	Basse
Écart à l'oracle	Routeur appris multi-signaux	Basse
Matériel unique	Réplication sur GPU, wattmètre	Basse
Exploration sur le développement	Pré-enregistrer la famille de politiques	Basse

TABLE 16 – Synthèse des points faibles et des améliorations proposées.

11 Conclusion

Nous sommes partis d'une question pratique : comment un assistant qui tourne sur une machine personnelle peut-il dépenser ses ressources avec discernement, en donnant à chaque question l'effort qu'elle mérite ? Pour y répondre, nous avons construit un benchmark et un protocole dont les critères de succès étaient fixés d'avance, établi des références solides, puis conçu une politique qui décide ensemble du modèle, du nombre de passages et de l'abstention, à partir de la suffisance des preuves et de la confiance du modèle.

Le résultat est en demi-teinte, et nous le rapportons comme tel. Sur le jeu de test, la politique préserve la qualité de la configuration de référence, commet moins d'erreurs et dépasse deux routeurs inspirés de la littérature. Mais elle ne réduit la latence que de 10 % et l'énergie de 9 %, au lieu des 20 % espérés. Nous en connaissons la cause : un seuil absolu, réglé sur un échantillon trop petit, qui s'est moins souvent déclenché sur des questions nouvelles.

Deux idées nous semblent survivre à ce résultat. Sur une machine locale, la qualité des preuves retrouvées est un guide plus sûr que la formulation de la question. Et savoir s'abstenir est ce qui permet d'économiser sans renoncer à la fiabilité. La suite du travail consiste à rendre ce levier stable : exprimer l'abstention comme un budget calibré, mieux juger la suffisance des preuves, et valider le tout sur un jeu de test plus vaste et entièrement neuf.

Reproductibilité. Le code, le benchmark et les empreintes de ses données, les configurations figées (étiquettes `protocole-v1.0` et `politique-v1.0`), les sorties brutes des trois passes et les mesures d'énergie sont publiés dans le dépôt indiqué en première page.

Références

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix : Automatically Mixing Language Models. arXiv preprint arXiv :2310.12963, 2023. URL https://arxiv.org/abs/2310.12963.

Simon Akesson and Frances A. Santos. Clustered Retrieved Augmented Generation (CRAG). arXiv preprint arXiv :2406.00029, 2024. URL https://arxiv.org/abs/2406.00029.

Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. LeanContext : Cost-efficient domain-specific question answering using LLMs. *Natural Language Processing Journal*, 2024. doi : 10.1016/j.nlp.2024.100065.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.11511.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT : How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv preprint arXiv :2305.05176, 2023. URL https://arxiv.org/abs/2305.05176.

Shengyuan Chen, Chuang Zhou, Zheng Yuan, Qinggang Zhang, Zeyang Cui, Hao Chen, Yilin Xiao, Jiannong Cao, and Xiao Huang. You Don't Need Pre-Built Graphs for RAG : Retrieval Augmented Generation with Adaptive Reasoning Structures. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026a. doi : 10.1609/aaai.v40i36.40278.

Yiwen Chen, Kuan Li, Fuzhen Zhuang, Deqing Wang, Zhao Zhang, Liwen Zhang, Yong Jiang, Shuai Wang, and Minhao Cheng. Route Before Retrieve : Activating Latent Routing Abilities of LLMs for RAG vs. Long-Context Selection. arXiv preprint arXiv :2605.10235, 2026b. URL https://arxiv.org/abs/2605.10235.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM : Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2404.14618.

Heming Fu, Shan Lin, Qianqian Xie, and Guojun Xiong. Not All Tokens Are Equal : Inflation-Aware Routing for Agentic LLM Systems. arXiv preprint arXiv :2608.13571, 2026. URL https://arxiv.org/abs/2608.13571.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models : A Survey. arXiv preprint arXiv :2312.10997, 2023. URL https://arxiv.org/abs/2312.10997.

Sai Hao, Hao Zeng, Hongxin Wei, and Bingyi Jing. RACER : Risk-Aware Calibrated Efficient Routing for Large Language Models. arXiv preprint arXiv :2603.06616, 2026. URL https://arxiv.org/abs/2603.06616.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench : A Benchmark for Multi-LLM Routing System. arXiv preprint arXiv :2403.12031, 2024. URL https://arxiv.org/abs/2403.12031.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG : Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of NAACL-HLT 2024*, 2024. URL https://arxiv.org/abs/2403.14403.

Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active Retrieval Augmented Generation. In *Proceedings of EMNLP 2023*, 2023. URL https://arxiv.org/abs/2305.06983.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language Models (Mostly) Know What They Know. arXiv preprint arXiv :2207.05221, 2022. URL https://arxiv.org/abs/2207.05221.

Rishi Kalra, Zekun Wu, Ayesha Gulley, Airlie Hilliard, Xin Guan, Adriano Koshiyama, and Philip Colin Treleaven. HyPA-RAG : A Hybrid Parameter Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications. In *Proceedings of the 1st Workshop on Customizable NLP (CustomNLP4U)*, 2024. doi : 10.18653/v1/2024.customnlp4u-1.18.

Amita Kamath, Robin Jia, and Percy Liang. Selective Question Answering under Domain Shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL https://arxiv.org/abs/2006.09462.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic Uncertainty : Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2302.09664.

Qiaomu Li, Qiuyuan Zhang, and Nong Ming. Beyond the Query : Do Retrieval Signals Improve Adaptive Multimodal RAG Routing ? arXiv preprint arXiv :2609.12437, 2026. URL https://arxiv.org/abs/2609.12437.

Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching Models to Express Their Uncertainty in Words. *Transactions on Machine Learning Research*, 2022. URL https://arxiv.org/abs/2205.14334.

Keming Lu, Hongyi Yuan, Runji Lin, Junyang Lin, Zheng Yuan, Chang Zhou, and Jingren Zhou. Routing to the Expert : Efficient Reward-guided Ensemble of Large Language Models. arXiv preprint arXiv :2311.08692, 2023. URL https://arxiv.org/abs/2311.08692.

Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. Estimating the Carbon Footprint of BLOOM, a 176B Parameter Language Model. arXiv preprint arXiv :2211.02001, 2022. URL https://arxiv.org/abs/2211.02001.

Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power Hungry Processing : Watts Driving the Cost of AI Deployment ? In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024. URL https://arxiv.org/abs/2311.16863.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When Not to Trust Language Models : Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL https://arxiv.org/abs/2212.10511.

Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT : Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of EMNLP 2023*, 2023. URL https://arxiv.org/abs/2303.08896.

Viktor Moskvoretskii, Maria Lysyuk, Mikhail Salnikov, Nikolay Ivanov, Sergey Pletenev, Daria Galimzianova, Nikita Krayko, Vasily Konovalov, Irina Nikishina, and Alexander Panchenko. Adaptive Retrieval Without Self-Knowledge ? Bringing Uncertainty Back Home. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. URL https://arxiv.org/abs/2501.12835.

Quang H. Nguyen, Thinh Dao, Duy C. Hoang, Juliette Decugis, Saurav Manchanda, Nitesh V. Chawla, and Khoa D. Doan. MetaLLM : A High-performant and Cost-efficient Dynamic Framework for Wrapping LLMs. arXiv preprint arXiv :2407.10834, 2024. URL https://arxiv.org/abs/2407.10834.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM : Learning to Route LLMs with Preference Data. arXiv preprint arXiv :2406.18665, 2024. URL https://arxiv.org/abs/2406.18665.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know What You Don't Know : Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018. URL https://arxiv.org/abs/1806.03822.

Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. From Words to Watts : Benchmarking the Energy Costs of Large Language Model Inference. arXiv preprint arXiv :2310.03003, 2023. URL https://arxiv.org/abs/2310.03003.

Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. DRAGIN : Dynamic Retrieval Augmented Generation based on the Information Needs of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2403.10081.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue : Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 2022. URL https://arxiv.org/abs/2108.00573.

Clovis Varangot-Reille, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. Doing More with Less : A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems. *Journal of Artificial Intelligence Research*, 2026. URL https://arxiv.org/abs/2502.00409.

Yile Wang, Peng Li, Maosong Sun, and Yang Liu. Self-Knowledge Guided Retrieval Augmentation for Large Language Models. arXiv preprint arXiv :2310.05002, 2023. URL https://arxiv.org/abs/2310.05002.

Ziqi Wang, Xi Zhu, Shuhang Lin, Haochen Xue, Minghao Guo, and Yongfeng Zhang. RAGRouter-Bench : A Dataset and Benchmark for Adaptive RAG Routing. arXiv preprint arXiv :2602.00296, 2026. URL https://arxiv.org/abs/2602.00296.

Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. Know Your Limits : A Survey of Abstention in Large Language Models. arXiv preprint arXiv :2407.18418, 2024. URL https://arxiv.org/abs/2407.18418.

Di Wu, Jia-Chen Gu, Kai-Wei Chang, and Nanyun Peng. Self-Routing RAG : Binding Selective Retrieval with Knowledge Verbalization. arXiv preprint arXiv :2504.01018, 2025. URL https://arxiv.org/abs/2504.01018.

Yichao Wu, Penghao Liang, Yafei Xiang, Mengwei Yuan, Jianan Liu, Jing Yang, Xianyou Li, and Weiran Yan. Tiny-Critic RAG : Empowering Agentic Fallback with Parameter-Efficient Small Language Models. arXiv preprint arXiv :2603.00846, 2026. URL https://arxiv.org/abs/2603.00846.

Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective Retrieval Augmented Generation. arXiv preprint arXiv :2401.15884, 2024. URL https://arxiv.org/abs/2401.15884.

Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, Lingkun Kong, Brian Moran, Jiaqi Wang, Yifan Ethan Xu, An Yan, Chenyu Yang, Eting Yuan, Hanwen Zha, Nan Tang, Lei Chen, Nicolas Scheffer, Yue Liu, Nirav Shah, Rakesh Wanga, Anuj Kumar, Wen tau Yih, and Xin Luna Dong. CRAG – Comprehensive RAG Benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2406.04744.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA : A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP 2018*, 2018. URL https://arxiv.org/abs/1809.09600.

Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. Large Language Model Cascades with Mixture of Thoughts Representations for Cost-efficient Reasoning. arXiv preprint arXiv :2310.03094, 2023. URL https://arxiv.org/abs/2310.03094.